



FUCHSIUS

Software Requirements Specification

Online Restaurant Ordering & Management System

1. Introduction

1.1 Purpose

This document specifies the functional and non-functional requirements for a full-stack restaurant website. The system allows visitors to browse restaurant information, view chef profiles, place food orders, and contact the restaurant, while giving administrators full control over all site content and operations through an admin panel.

1.2 Scope

The system consists of a public-facing website (Home, About, Chef Details, Order Process, Contact pages) and a secured Admin Panel with access to every feature and content area of the site, including image and content management for all pages.

1.3 Technology Stack

- Frontend: React.js
- Backend: Node.js (Express)
- ORM: Prisma
- Database (Production): MySQL
- Database (Testing): SQLite
- Architecture: REST API - React client communicating with a Node/Express + Prisma backend

2. Overall Description

2.1 User Classes

- Visitor / Customer — browses pages and places orders
- Administrator — manages all content, images, orders and site data via the Admin Panel

2.2 Operating Environment

The application will run as a responsive web app accessible via modern desktop and mobile browsers, with the backend deployed as a Node.js REST API connected to a MySQL database through Prisma ORM.

3. Functional Requirements - Public Website

3.1 Home Page

ID	Requirement
FR-1.1	Display restaurant banner/hero image, name, tagline and a short introduction.
FR-1.2	Show featured/highlighted dishes with images, names and prices.
FR-1.3	Provide navigation links to Home, About, Chef Details, Order, and Contact pages.
FR-1.4	Display promotional images/banners managed from the Admin Panel.

3.2 About Page

ID	Requirement
FR-2.1	Display restaurant history, mission and story with supporting images.
FR-2.2	Show restaurant location, opening hours and gallery images.
FR-2.3	All text and images on this page must be editable from the Admin Panel.

3.3 Chef Details Page

ID	Requirement
FR-3.1	List all chefs with photo, name, designation/title and short bio.
FR-3.2	Show each chef's specialty dishes or cuisine expertise.
FR-3.3	Admin can add, edit, delete and reorder chef profiles, including chef images.

3.4 Order Process Page

ID	Requirement
FR-4.1	Display the food menu with categories, item images, descriptions and prices.
FR-4.2	Allow customers to add items to a cart, adjust quantity, and remove items.
FR-4.3	Allow customers to enter delivery/contact details and place an order.
FR-4.4	Show an order summary and confirmation (order ID, items, total) after checkout.
FR-4.5	Persist orders to the database via the Node/Prisma backend for admin tracking.

3.5 Contact Page

ID	Requirement
FR-5.1	Provide a contact form (name, email, message) that submits to the backend.
FR-5.2	Display restaurant address, phone number, email and an embedded map.
FR-5.3	Admin can view and manage submitted contact messages.

4. Functional Requirements - Admin Panel

The Admin Panel provides authenticated administrators with full access to every feature and content area of the system. A single administrator role can create, read, update and delete all data described below.

ID	Requirement
FR-6.1	Secure admin login (authentication) protecting all admin routes.
FR-6.2	Manage Home page content: banners, featured dishes, promotional images.
FR-6.3	Manage About page content: text sections and gallery images.
FR-6.4	Manage Chef Details: add/edit/delete chefs, photos, bios and specialties.
FR-6.5	Manage Menu & Orders: add/edit/delete menu items, categories, prices and images; view, update status of, and manage all customer orders.
FR-6.6	Manage Contact submissions: view and respond to/delete customer messages; edit contact info and map details.
FR-6.7	Manage Images: upload, replace and delete images used across all public pages, stored via the backend and referenced in the database.
FR-6.8	Dashboard overview summarizing recent orders, messages and content status.

5. Non-Functional Requirements

ID	Requirement
NFR-1	The system shall be built with React (frontend) and Node.js/Express (backend).
NFR-2	Data access shall go through Prisma ORM against a SQL database in production.
NFR-3	Automated/integration testing shall use SQLite as the Prisma test database.
NFR-4	The UI shall be responsive across desktop, tablet and mobile screen sizes.
NFR-5	Admin routes and actions shall require authentication/authorization.
NFR-6	Page load and API response times should support a smooth browsing/ordering experience.

6. Data Requirements (High-Level)

- User/Admin — id, name, email, password (hashed), role
- Chef — id, name, title, bio, specialty, image
- MenuItem — id, name, description, price, category, image
- Order — id, customer info, items (line items), total, status, createdAt
- ContactMessage — id, name, email, message, createdAt
- PageContent/Image — id, page reference, content/image, updatedAt

7. Assumptions & Constraints

- Payment gateway integration is out of scope unless specified separately; order process covers cart, checkout details and order confirmation.
- SQLite is used only for the testing environment; the production database is a standard MySQL engine accessed through Prisma.
- All images referenced across pages are managed and stored through the Admin Panel.